TD 3.1

Exercice 1

Expliquez les lignes suivantes du syslog.conf :

mail.notice /var/log/mail

Les messages de priorité notice et supérieurs de la « facilité » mail sont envoyés dans le fichier mail

*.emerg @mothership.mydomain.org

Les messages de priorité emerg et supérieurs de toutes les « facilités » vers la machine mothership

mail,uucp.notice /var/log/mail

Les messages de priorité notice et supérieurs des « facilités » mail et uucp sont envoyés dans le fichier mail

mail,uucp.notice;uucp.!=alert /var/log/mail

Les messages de priorité notice et supérieurs des « facilités » mail et uucp sauf de priorité alert pour uucp sont envoyés dans le fichier mail

mail,uucp.notice;uucp.!alert /var/log/mail

Les messages de priorité notice et supérieurs des « facilités » mail et uucp sauf de priorité alert et supérieure pour uucp sont envoyés dans le fichier mail

Exercice 2 - Rotation des journaux

1. Configurez la rotation automatique du fichier /var/log/warnings.log pour éviter qu'il devienne trop volumineux (utilisez logrotate).

2. Limitez la taille du fichier journal à 10 Mo et conservez les 5 derniers fichiers.

Exercice 2 - Rotation des journaux

1. Configurez la rotation automatique du fichier /var/log/warnings.log pour éviter qu'il devienne trop volumineux (utilisez logrotate).
2. 2. Limitez la taille du fichier journal à 10 Mo et conservez les 5 derniers fichiers.

sudo nano /etc/logrotate.d/warnings

Ajouter

/var/log/warnings.log {
size 10M
rotate 5
compress
missingok
notifempty
create 640 root root }

Pour tester

sudo logrotate -d /etc/logrotate.conf

Appliquer la rotation

sudo logrotate /etc/logrotate.conf

Syslog et Rsyslog

syslog et **rsyslog** sont deux systèmes de gestion et de traitement des journaux sous Unix/Linux. Ils partagent des fonctionnalités de base, mais diffèrent par leurs capacités et leur utilisation.

Rsyslog

rsyslog est une version améliorée et enrichie de syslog, introduite pour combler ses limitations et répondre aux besoins modernes.

- **Compatibilité ascendante** : Peut utiliser les configurations de syslog (/etc/syslog.conf), mais utilise généralement /etc/rsyslog.conf.
- **Protocole avancé** :
  - Supporte les protocoles Syslog modernes comme RFC 5424 (plus robuste que RFC 3164).
  - Utilise UDP, TCP, et TLS pour une communication sécurisée.
- **Puissance et extensibilité** :
  - Capable de traiter de très grands volumes de journaux.
  - Peut écrire les journaux dans des bases de données (MySQL, PostgreSQL) ou les envoyer à des systèmes centralisés.
- **Filtres avancés** :
  - Supporte des règles complexes basées sur des expressions régulières ou des conditions spécifiques.

Syslog et Rsyslog

- **Modules supplémentaires** :
  - Peut ajouter des fonctionnalités via des modules (ex. ommysql pour MySQL).
- **Performance améliorée** :
  - Optimisé pour les environnements à forte intensité de journalisation.
- **Journalisation structurée** :
  - Supporte des formats comme JSON pour des intégrations modernes.
- **Gestion centralisée** :
  - Très utilisé pour la collecte centralisée des journaux dans des environnements distribués.

Syslog et Rsyslog

- **Quand utiliser l'un ou l'autre ?**
  - **Syslog** :
    - Si vous avez un système simple et léger, où les besoins de journalisation sont basiques.
  - **Rsyslog** :
    - Si vous avez un environnement complexe, de grands volumes de journaux, ou un besoin d'intégration avec des bases de données et des systèmes modernes

*Dans la plupart des cas modernes, **rsyslog** est préféré car il est plus puissant et flexible tout en restant compatible avec **syslog**.*

Résumé des différences

Caractéristique	syslog	rsyslog
Complexité	Simple	Avancé et modulaire
Protocoles supportés	UDP (RFC 3164)	UDP, TCP, TLS (RFC 5424)
Base de données	Non	Oui (MySQL, PostgreSQL)
Gestion centralisée	Basique	Avancée
Filtres	Basique	Avancés (regex, conditions)
Extensibilité	Faible	Forte (modules, plugins)
Performance	Moyen	Haute
Formats supportés	Texte brut	JSON, Texte brut, etc.

Syslog et Rsyslog

- **Tester le traitement des messages**

Supposant que les messages sont enregistrés dans ***/var/log/auth_warning.log*** lorsqu'ils sont interceptés.

On peut générez un message de journal avec la commande suivante :

```
logger -p auth.warning "Test warning message"
```

On peut alors vérifiez que le message a été traité et ajouté au fichier /var/log/auth_warning.log

```
cat /var/log/auth_warning.log
```

Exercice 3 - Alertes en temps réel pour des événements critiques

Configurer une alerte en temps réel pour tout événement critique (crit ou supérieur) sur le système, de manière à envoyer un email automatique à un administrateur avec le contenu du journal.

1. Configurez **rsyslog** pour détecter les événements critiques et exécuter un script personnalisé pour envoyer un email.
2. Implémentez un script shell pour générer l'email avec les informations pertinentes.

Indications :

- Utilisez le module **omprog** ou configurez une commande via une directive dans /etc/rsyslog.conf.
- Assurez-vous que le système peut envoyer des emails (par exemple, avec **mailx** ou **sendmail**).

Qu'est-ce que omprog dans rsyslog ?
*omprog est un **module de sortie (output module)** de **rsyslog** qui permet d'exécuter un programme externe pour traiter ou enregistrer les messages de journal. Au lieu d'écrire les messages dans un fichier, une base de données ou une autre destination standard, ce module envoie les messages à un programme tiers pour un traitement personnalisé.*

Il permet d'écrire	mail.notice	| /usr/local/bin/send_notice.sh
Au lieu d'envoyer simplement vers un fichier	mail.notice	/var/log/mail

Exercice 3 - Alertes en temps réel pour des événements critiques

Configurer une alerte en temps réel pour tout événement critique (crit ou supérieur) sur le système, de manière à envoyer un email automatique à un administrateur avec le contenu du journal.

1. Configurez **rsyslog** pour détecter les événements critiques et exécuter un script personnalisé pour envoyer un email.
2. Implémentez un script shell pour générer l'email avec les informations pertinentes.

Dans /etc/rsyslog.conf, ajoutez : *.crit |/usr/local/bin/send_alert.sh

Créez un script /usr/local/bin/send_alert.sh :

```
#!/bin/bash
while read line; do
   echo "$line" | mailx -s "Alerte critique détectée" admin@example.com
done
```

Rendez-le exécutable : chmod +x /usr/local/bin/send_alert.sh

Redémarrez rsyslog pour appliquer la configuration :

sudo systemctl restart rsyslog ou
sudo service rsyslog restart ou
sudo killall rsyslogd

Exercice 4 - Configurer rsyslog pour envoyer des journaux à un serveur distant

Configurer un serveur Unix pour envoyer tous les messages de journal à un serveur distant ayant l'adresse IP 192.168.1.100 sur le port UDP **514**.

Exercice 4 - Configurer rsyslog pour envoyer des journaux à un serveur distant

Configurer un serveur Unix pour envoyer tous les messages de journal à un serveur distant ayant l'adresse IP 192.168.1.100 sur le port UDP **514**.

Ouvrez /etc/rsyslog.conf sur le client et ajoutez la ligne suivante pour envoyer les journaux au serveur distant :

. @192.168.1.100:514

Si vous voulez utiliser TCP (plus fiable qu'UDP qui ne garanti pas la livraison des messages) pour plus de fiabilité, utilisez @@ au lieu de @ :

. @@192.168.1.100:514

Exercice 5 - Filtrer les messages de niveau “warning” et les transmettre à un script Bash pour traitement

Configurer **rsyslog** pour détecter tous les messages de niveau **warning** (ou supérieur) provenant de la facilité auth et les envoyer à un script Bash. Ce script doit enregistrer les messages dans un fichier /var/log/auth_warning.log avec un horodatage du type : ***2024-12-30 14:35:42 Authentication failure for user 'admin’***.

Exercice 5 - Filtrer les messages de niveau “warning” et les transmettre à un script Bash pour traitement

Configurer **rsyslog** pour détecter tous les messages de niveau **warning** (ou supérieur) provenant de la facilité auth et les envoyer à un script Bash. Ce script doit enregistrer les messages dans un fichier /var/log/auth_warning.log avec un horodatage du type : ***2024-12-30 14:35:42 Authentication failure for user 'admin’***.

Créez un fichier de script nommé /usr/local/bin/process_auth_warnings.sh :

```
#!/bin/bash
while read -r line; do
   echo "$(date '+%Y-%m-%d %H:%M:%S') $line" >> /var/log/auth_warning.log
Done
```

Rendez-le exécutable :

```
sudo chmod +x /usr/local/bin/auth_warnings.sh
```

Modifiez le fichier /etc/rsyslog.conf :

```
auth.warning |/usr/local/bin/auth_warnings.sh
Ou
if ($syslogfacility-text == 'auth' and $msg contains 'warning') then |/usr/local/bin/auth_warnings.sh
Ou
if ($syslogfacility-text == 'auth' and $syslogseverity <= 4) then {
   action(type="omprog" binary="/usr/local/bin/auth_warnings.sh")
   stop }
```

Exercice 5 – Surveiller les journaux pour un utilisateur

Créer un script qui surveille des journaux pour un utilisateur spécifique

1. Vérifie si un utilisateur donné existe sur le système.
2. Recherche dans les journaux (/var/log/syslog) des événements contenant le nom de l'utilisateur.
3. Enregistre les résultats dans un fichier texte situé dans le répertoire personnel de l'utilisateur.
4. Planifie l'exécution du script quotidiennement avec cron.

Exercice 5 – Surveiller les journaux pour un utilisateur

```bash
#!/bin/bash

# Demander le nom de l'utilisateur
echo "Entrez le nom de l'utilisateur à surveiller :"
read -r USERNAME

# Vérifier si l'utilisateur existe
if ! id "$USERNAME" &>/dev/null; then
   echo "L'utilisateur $USERNAME n'existe pas."
   exit 1
fi

# Répertoire de l'utilisateur
USER_HOME=$(getent passwd "$USERNAME" | cut -d: -f6)
# getent passwd "$USERNAME" recherche l'entrée correspondant à l'utilisateur $USERNAME
# dans la base de données des utilisateurs (login:passwd:uid:gid:gecos:home dir:shell)

# Vérifier si le répertoire utilisateur existe
if [ ! -d "$USER_HOME" ]; then
   echo "Le répertoire de l'utilisateur $USERNAME est introuvable."
   exit 1
fi

# Fichier pour enregistrer les événements
EVENTS_FILE="$USER_HOME/user_events.log"

# Rechercher les événements contenant le nom de l'utilisateur dans /var/log/syslog
echo "Événements pour l'utilisateur $USERNAME (exécuté le $(date)) :" > "$EVENTS_FILE"
grep "$USERNAME" /var/log/syslog >> "$EVENTS_FILE"

echo "Les événements ont été enregistrés dans $EVENTS_FILE"
```

```
crontab –e

0 0 * * * /home/utilisateur/scripts/monitor_user_simple.sh > /home/utilisateur/scripts/monitor_user_simple.log 2>&1
```